



ค่ายโอลิมปิกวิชาการ ครั้งที่ 18
ศูนย์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Greedy algorithm
อัลกอริทึมเชิงละโมภ

ดร.ณัฐกิตติ์ จิตรเอื้อตระกูล

nattagit.j@sci.kmutnb.ac.th

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

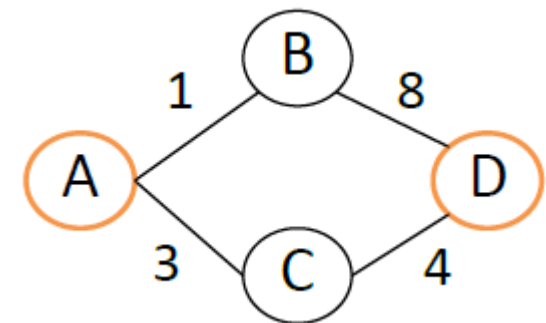
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

อัลกอริทึมเชิงละโมภ (Greedy algorithm)

- อัลกอริทึมเชิงละโมภเป็นเทคนิคที่เหมาะสมในการแก้ปัญหาเพื่อค้นหาคำตอบที่เหมาะสมที่สุด (optimal solution)
- หลักการเชิงละโมภ ก็คือการลดเวลาโดยการตัดสินใจเลือกคำตอบย่อย (sub solutions) ที่ดีที่สุดในขณะนั้น โดยที่คำตอบที่เลือกจะต้องไม่ขัดแย้งกับเงื่อนไขบังคับของปัญหา (problem constraint) โดยคาดหวังว่าคำตอบที่เลือกจะได้คำตอบที่เหมาะสมที่สุด
- ดังนั้นประสิทธิภาพของ Greedy จึงจะขึ้นอยู่กับ เกณฑ์ที่นำไปใช้ในการตัดสินใจ (greedy criterion) ในการเลือกแต่ละคำตอบย่อย

Greedy Algorithm

- ข้อดีของอัลกอริทึมเชิงละโมภ
 - การพัฒนาโปรแกรมทำได้ง่าย
 - อัลกอริทึม greedy มักให้เวลาในการทำงานที่ดี
บ่อยครั้งเป็นฟังก์ชัน polynomial
- ข้อเสีย
 - ไม่รับประกันว่าคำตอบที่ได้จะเป็นคำตอบที่ดีที่สุด (optimal solution) เสมอ เนื่องจากการเลือกคำตอบย่อยที่ดีที่สุดทุกครั้งอาจไม่ได้นำไปสู่คำตอบที่ดีที่สุดเสมอไป



ปัญหาการแลกเหรียญ (Coin changing problem)

- แคชเชียร์ 7-11 มีเหรียญ 1 บาท 2 บาท 5 บาท และ 10 บาท จำนวนไม่จำกัด ต้องการทอนเงิน 57 บาท อยากทราบว่าจะต้องใช้เหรียญเหล่านี้ในการทอนเงินให้พอดีน้อยที่สุดเท่าไร



เกณฑ์การเลือกของ Greedy

1. ทอนเหรียญที่มีมูลค่ามากที่สุดก่อนให้มากที่สุดเท่าที่จะเป็นไปได้
2. หากเงินมีค่าน้อยกว่ามูลค่าเหรียญแล้ว ให้ทอนเหรียญที่มีมูลค่ารองลงมา
3. ทำซ้ำเหมือนเดิมกับเหรียญที่มีมูลค่าน้อยลง จนกระทั่งเงินทอนมีค่าเป็นศูนย์หรือไม่สามารถทอนเงินได้อีกต่อไป



อัลกอริทึม Greedy กับ Coin changing

พัฒนาอัลกอริทึม greedy ในการทอนเหรียญให้น้อยที่สุด โดยใช้เกณฑ์ในการเลือกตามตัวอย่างที่ผ่านมา กำหนดให้อินพุตคือ อาร์เรย์ของมูลค่าเหรียญ $D[] = \{1, 5, 2, 10\}$ และ W คือจำนวนเงินที่ต้องการทอน

Code Coin changing with Greedy

```
int coin change (int D[], int n, int W)
{   int num = 0;           // number of returned coins
    int i=0;
    sort (D, n);           // sort coins by descending
    while ((W > 0) && (i < n))
    {   int coin = W/D[i];
        num += coin;
        W = W-coin*D[i++];
    }
    return num;
}
```

ปัญหาเหมาะสำหรับอัลกอริทึมเชิงละโมบ

ตัวอย่าง

- ปัญหา knapsack
- ปัญหาการจัดตารางงาน (task scheduling)
- ปัญหาการบีบอัดข้อความ (text compression)

 1	 4	 5	 6
 3	 4	 1	 3
 3	 20	 10	 6
 8	 8	 2	 3

ปัญหา 0/1 Knapsack

- เลือกของ K ชิ้นใส่กระเป๋าให้มีกำไร (profit) มากที่สุด โดยกระเป๋าไม่ขาด

Item	Weight	Profit
O1	18	25
O2	15	24
O3	10	15
O4	5	8



Weight limit

25

ปัญหา 0/1 Knapsack

คำตอบของปัญหา 0/1 Knapsack สามารถแสดงในรูปของไบนารีเซตโดยที่ไอเท็มซึ่งถูกเลือก ('1') และไม่ถูกเลือก ('0')

Item	Weight	Profit
O1	18	25
O2	15	24
O3	10	15
O4	5	8

X	Profit	Weight
{0, 0, 1, 1}	$15 + 8 = 23$	$10 + 5 = 15$

เกณฑ์การตัดสินใจ greedy

สมมติจำนวนของ $N = 4$ และน้ำหนักกระเป๋าที่รับได้คือ $M = 25$ กำหนดให้ของแต่ละชิ้นมีมูลค่า $(p_1, p_2, p_3, p_4) = (25, 24, 15, 8)$ และน้ำหนักคือ $(w_1, w_2, w_3, w_4) = (18, 15, 10, 5)$

1. เลือกของน้ำหนักน้อยที่สุดใส่ก่อน (Minimize weight)
2. เลือกของที่มีมูลค่ามากที่สุดใส่ก่อน (Maximize Profit)

ตัวอย่าง Greedy สำหรับ **0/1 Knapsack**

สมมติ $n = 4$ และน้ำหนักถุงที่รับได้คือ $M=25$ กำหนดให้ของแต่ละชิ้นมีมูลค่า

$(p_1, p_2, p_3, p_4) = \{25, 24, 15, 8\}$ และน้ำหนักคือ $(w_1, w_2, w_3, w_4) = \{18, 15, 10, 5\}$

O1	O2	O3	O4
$25/18 = 1.39$	$24/15 = 1.6$	$15/10 = 1.5$	$8/5 = 1.6$

ใส่ของลงไปในถุงตามลำดับ : o2 -> o4 -> o3 -> o1

(x_1, x_2, x_3, x_4)	Profit	Weight
0 1 0 1	$24+8 = 32$	$15+5 = 20$

A good strategy for 0/1 knapsack

- เลือกสิ่งของโดยนำของซึ่งมีมูลค่ามากแต่น้ำหนักน้อยใส่ถุงก่อน
- กำหนดให้ความหนาแน่น (**density**) คือสัดส่วนของมูลค่าและน้ำหนักของวัตถุ ตัวอย่างเช่น วัตถุ A มีมูลค่า 25 น้ำหนัก 18 จะมีความหนาแน่นคือ $25/18 = 1.39$
- หลักการของ Greedy
 - เรียงลำดับของแต่ละชิ้นตามความหนาแน่นของมันจากน้อยไปมาก
 - สำหรับแต่ละขั้นตอน เลือกของที่มีความหนาแน่นมากใส่ลงไปก่อนจนกระทั่งไม่สามารถใส่ได้ (ถุงขาด)

Greedy solutions

1. เรียงลำดับสิ่งของตามน้ำหนักที่น้อยที่สุดก่อน

o4->o3->o2->o1

$X = \{0, 0, 1, 1\}$ $P = 23$

2. เรียงลำดับสิ่งของตามมูลค่าจากมากไปน้อย

o1->o2->o3->o4

$X = \{1, 0, 0, 1\}$ $P = 33$

Item	Weight	Profit
O1	18	25
O2	15	24
O3	10	15
O4	5	8

M = 25

ปัญหา Fractional Knapsack

- ปัญหา 0/1 Knapsack อนุญาตให้ใส่ของลงไปในถุงทั้งวัตถุ ซึ่งมักจะทำให้ถุงเหลือน้ำหนักบางส่วนทิ้งไว้ ตัวอย่างเช่น ในตัวอย่างที่แล้วเราจะเหลือน้ำหนัก 5 หน่วยที่ไม่สามารถใส่ของได้แล้ว
- ปัญหา Fractional knapsack จะเพิ่มเงื่อนไขของปัญหาคือว่าเราสามารถแบ่งของเพื่อนำไปใส่ในถุงได้ แต่มูลค่าของสิ่งนั้นจะลดลงตามสัดส่วนที่แบ่งด้วยเช่นกัน

	O1	O2	O3	O4
Profit	25	24	15	8
Weight	18	15	10	5
Density	1.39	1.6	1.5	1.6

(x1, x2, x3, x4)

1/4 0 1/2 1

Profit

$$(25 \cdot 1/4) + (15 \cdot 1/2) + 8 = 21.75$$

Weight

$$(15 \cdot 1/4) + (5 \cdot 1/2) + 5 = 11.25$$

Code Fractional Knapsack

```
double fractionKP(int W, struct Item t[], int n)
{  sort(t, n);                // ใช้ฟังก์ชันจัดเรียงของในกระเป๋า
  int curWeight = 0;          // current weight in knapsack
  double finalval = 0.0;      // result
  for(int i=0; i<n; i++)
  {  if(curWeight + t[i].weight <= W)    // if we can add item
    {  curWeight += t[i].weight;
      finalval += t[i].value;
    }
  }
```

```
struct Item
{  int value;
  int weight;
};
```

Code Fractional Knapsack

```
else // if we can't add current item
{   int remain = W - curWeight;
    finalvalue += t[i].value*((double)remain/t[i].weight);
    break;
}
}
return finalval;
}
```

Greedy strategy for fractional knapsack

วิธีการเลือกตามความหนาแน่นของวัตถุที่ใช้ในปัญหา 0/1 Knapsack จะยังให้คำตอบที่เป็น Optimal หรือไม่?

O1	O2	O3	O4
$25/18 = 1.39$	$24/15 = 1.6$	$15/10 = 1.5$	$8/5 = 1.6$

ใส่ของลงไปในถุงตามลำดับ : o2 -> o4 -> **o3** -> o1

(x1, x2, x3, x4)

0 1 **1/2** 1

Profit

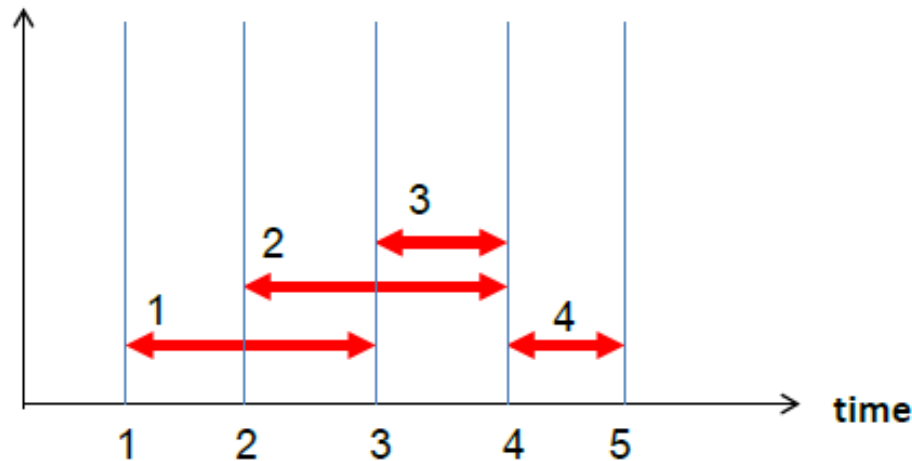
$24 + (15 * \mathbf{1/2}) + 8 = 39.5$

Weight

$15 + 5 + (10 * \mathbf{1/2}) = 25$

ปัญหาการจัดตารางงาน (task scheduling)

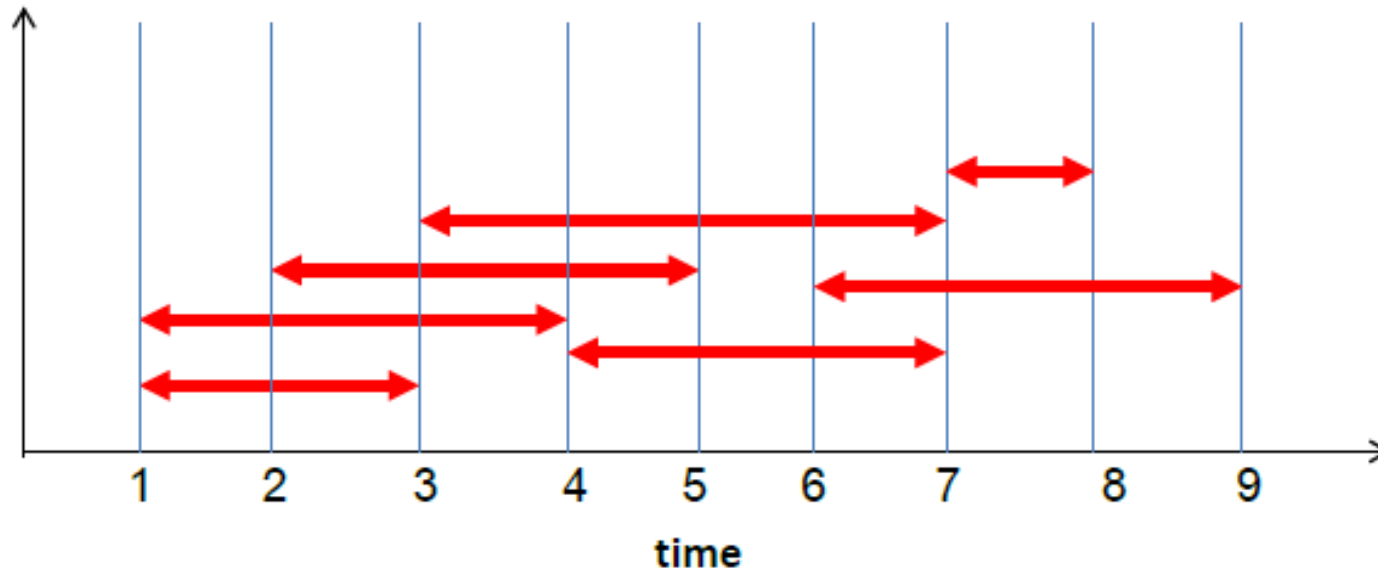
- กำหนดมีเซตของงานทั้งสิ้น n งาน ซึ่งรอที่จะถูกประมวลผลโดย machine แต่ละงานจะประกอบไปด้วย (s_i, f_i) ซึ่งหมายถึงเวลาเริ่มต้นและเวลาสิ้นสุดของการประมวลผลงานที่ i เนื่องจาก machine สามารถประมวลผลได้เพียงครั้งละ 1 งาน ดังนั้นหาก 2 งานมีเวลาซ้อนทับกัน (overlap time) machine จะต้องเลือก 1 งานมาประมวลผลและจะต้องยกเลิกอีกงานไป



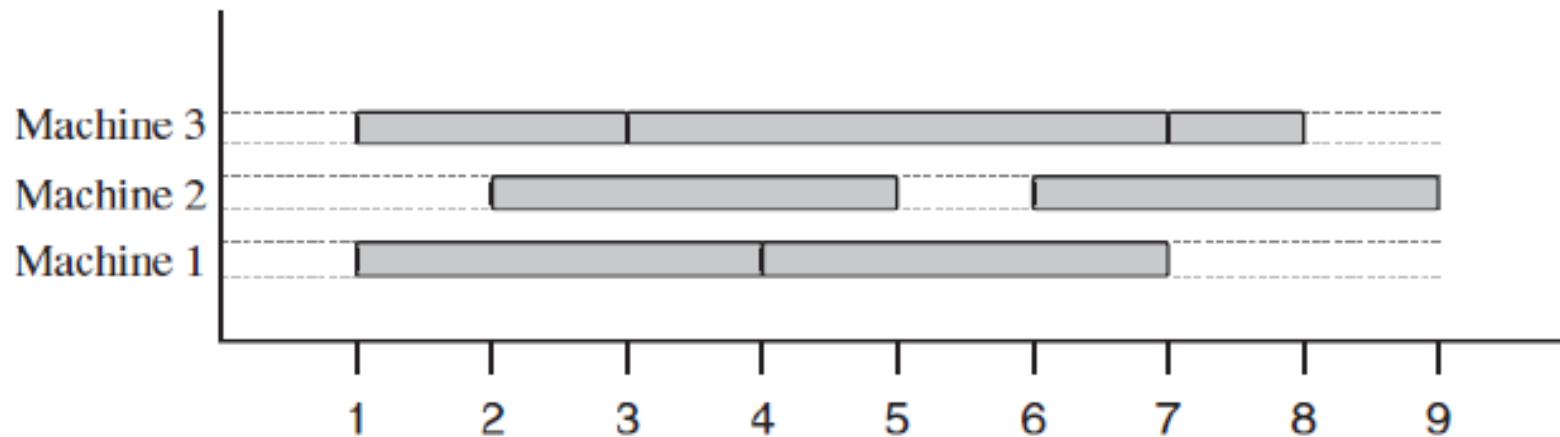
ประเด็นคือต้องการหาจำนวน machine ที่น้อยที่สุดเพื่อรองรับการประมวลผลงานทั้งหมด

หากเลือกประมวลผลงาน 2 จะต้องยกเลิกงาน 1 และงาน 3 หรือต้องการ machine เพิ่มอีก 1 ตัว เพื่อประมวลผลงาน 1 และงาน 3 ตามลำดับ

ตัวอย่าง



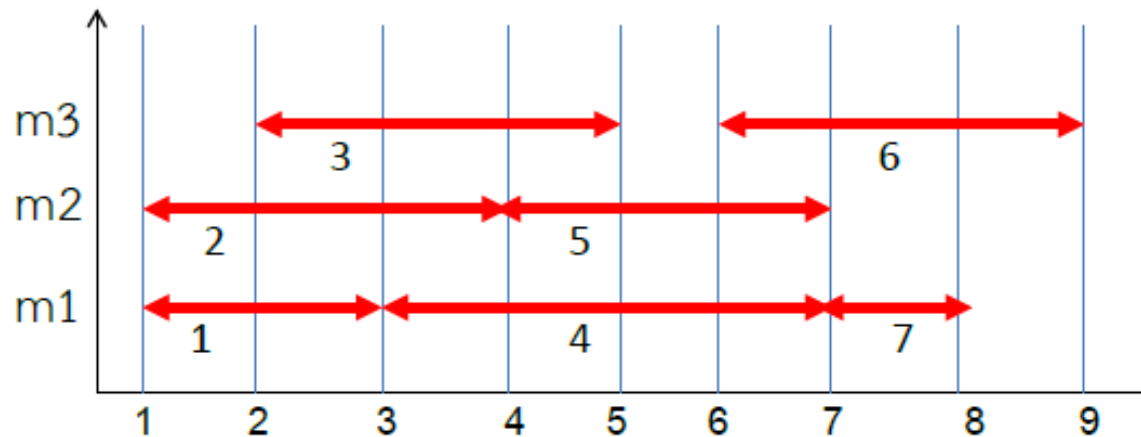
จะต้องใช้ machine อย่างน้อยที่สุด
3 ตัวในการประมวลผลงานทั้งหมด
โดยไม่ขัดแย้ง



Longest-first strategy

เลือกงานที่ใช้เวลาในการประมวลผลนานที่สุดก่อน

- เรียงลำดับงานตามเวลาที่ใช้ในการประมวลผลจากมากไปน้อย
- หากเกิดการทับซ้อนเวลาก็เพิ่ม machine อีก 1 ตัว



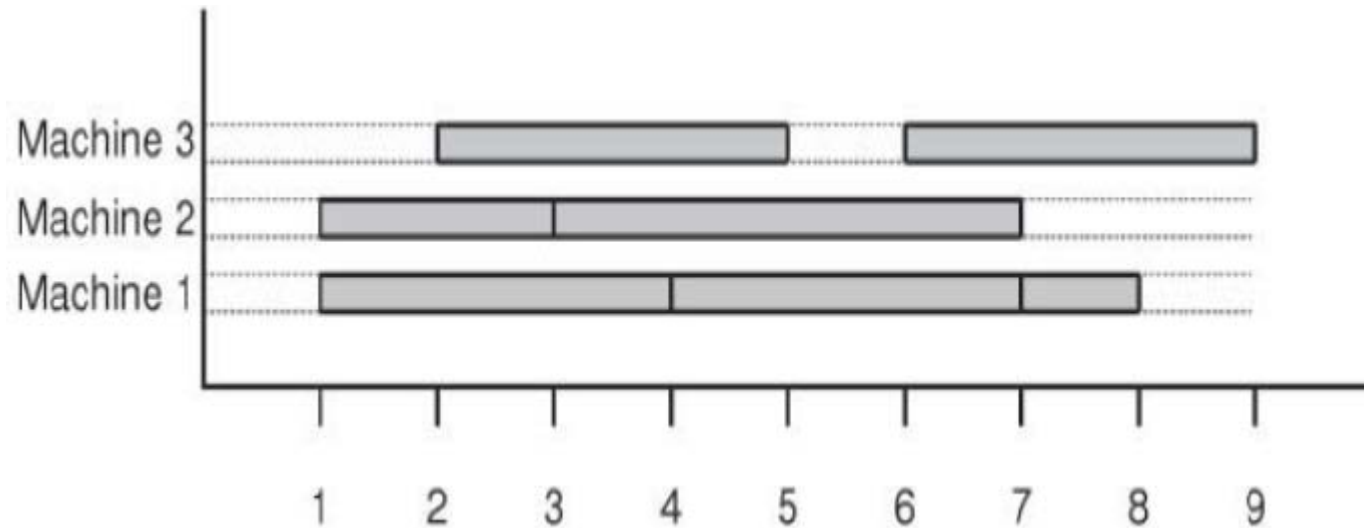
จะต้องใช้อย่างน้อย 3 machine

Task	s_i	f_i	rank
1	1	3	6
2	1	4	2
3	2	5	3
4	3	7	1
5	4	7	4
6	6	9	5
7	7	8	7

Earliest start-time strategy

เลือกงานที่มีเวลาเริ่มต้น (s_i) ก่อน

- เรียงลำดับงานตามเวลาเริ่มต้นจากน้อยไปมาก
- หากเกิดการทับซ้อนเวลาก็เพิ่ม machine อีก 1 ตัว



จะต้องใช้อย่างน้อย 3 machine

Task	s_i	f_i	rank
1	1	3	1
2	1	4	2
3	2	5	3
4	3	7	4
5	4	7	5
6	6	9	6
7	7	8	7

Other strategies for job scheduling

- Earliest finish-time strategy
เลือกงานที่มีเวลาสิ้นสุดก่อน
- Smallest interval strategy
เลือกงานที่ใช้เวลาประมวลผลน้อยที่สุดก่อน
- Least conflict strategy
เลือกงานที่มีช่วงเวลา conflict กับงานอื่นน้อยที่สุดก่อน 26

Text Compression

ปกติแล้วคอมพิวเตอร์จัดเก็บไฟล์ในลักษณะที่เป็น fixed-length binary code ตัวอย่างเช่นสมมติไฟล์หนึ่งประกอบไปด้วยตัวอักษรและความถี่ที่ใช้งานดังนี้

Character	Code	Frequency
a	0110 0001	10
e	0110 0101	15
i	0110 1001	12
s	0111 0011	3
t	0111 0100	4
r	0111 0010	1

TOTAL $45 * 8 = 360$ bits

ASCII Code

Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII
0	00000000	000	00	NUL	32	00100000	040	20	SP	64	01000000	100	40	@	96	01100000	140	60	`
1	00000001	001	01	SOH	33	00100001	041	21	!	65	01000001	101	41	A	97	01100001	141	61	a
2	00000010	002	02	STX	34	00100010	042	22	"	66	01000010	102	42	B	98	01100010	142	62	b
3	00000011	003	03	ETX	35	00100011	043	23	#	67	01000011	103	43	C	99	01100011	143	63	c
4	00000100	004	04	EOT	36	00100100	044	24	\$	68	01000100	104	44	D	100	01100100	144	64	d
5	00000101	005	05	ENQ	37	00100101	045	25	%	69	01000101	105	45	E	101	01100101	145	65	e
6	00000110	006	06	ACK	38	00100110	046	26	&	70	01000110	106	46	F	102	01100110	146	66	f
7	00000111	007	07	BEL	39	00100111	047	27	'	71	01000111	107	47	G	103	01100111	147	67	g
8	00001000	010	08	BS	40	00101000	050	28	(	72	01001000	110	48	H	104	01101000	150	68	h
9	00001001	011	09	HT	41	00101001	051	29	)	73	01001001	111	49	I	105	01101001	151	69	i
10	00001010	012	0A	LF	42	00101010	052	2A	*	74	01001010	112	4A	J	106	01101010	152	6A	j
11	00001011	013	0B	VT	43	00101011	053	2B	+	75	01001011	113	4B	K	107	01101011	153	6B	k
12	00001100	014	0C	FF	44	00101100	054	2C	,	76	01001100	114	4C	L	108	01101100	154	6C	l
13	00001101	015	0D	CR	45	00101101	055	2D	-	77	01001101	115	4D	M	109	01101101	155	6D	m
14	00001110	016	0E	SO	46	00101110	056	2E	.	78	01001110	116	4E	N	110	01101110	156	6E	n
15	00001111	017	0F	SI	47	00101111	057	2F	/	79	01001111	117	4F	O	111	01101111	157	6F	o
16	00010000	020	10	DLE	48	00110000	060	30	0	80	01010000	120	50	P	112	01110000	160	70	p
17	00010001	021	11	DC1	49	00110001	061	31	1	81	01010001	121	51	Q	113	01110001	161	71	q
18	00010010	022	12	DC2	50	00110010	062	32	2	82	01010010	122	52	R	114	01110010	162	72	r
19	00010011	023	13	DC3	51	00110011	063	33	3	83	01010011	123	53	S	115	01110011	163	73	s
20	00010100	024	14	DC4	52	00110100	064	34	4	84	01010100	124	54	T	116	01110100	164	74	t
21	00010101	025	15	NAK	53	00110101	065	35	5	85	01010101	125	55	U	117	01110101	165	75	u
22	00010110	026	16	SYN	54	00110110	066	36	6	86	01010110	126	56	V	118	01110110	166	76	v
23	00010111	027	17	ETB	55	00110111	067	37	7	87	01010111	127	57	W	119	01110111	167	77	w
24	00011000	030	18	CAN	56	00111000	070	38	8	88	01011000	130	58	X	120	01111000	170	78	x
25	00011001	031	19	EM	57	00111001	071	39	9	89	01011001	131	59	Y	121	01111001	171	79	y
26	00011010	032	1A	SUB	58	00111010	072	3A	:	90	01011010	132	5A	Z	122	01111010	172	7A	z
27	00011011	033	1B	ESC	59	00111011	073	3B	;	91	01011011	133	5B	[	123	01111011	173	7B	{
28	00011100	034	1C	FS	60	00111100	074	3C	<	92	01011100	134	5C	\	124	01111100	174	7C	
29	00011101	035	1D	GS	61	00111101	075	3D	=	93	01011101	135	5D	]	125	01111101	175	7D	}
30	00011110	036	1E	RS	62	00111110	076	3E	>	94	01011110	136	5E	^	126	01111110	176	7E	~
31	00011111	037	1F	US	63	00111111	077	3F	?	95	01011111	137	5F	_	127	01111111	177	7F	DEL

Data Types in C

Type	Size (bits)	Size (bytes)	Range
char	8	1	-128 to 127
unsigned char	8	1	0 to 255
int	16	2	-2^{15} to $2^{15}-1$
unsigned int	16	2	0 to $2^{16}-1$
short int	8	1	-128 to 127
unsigned short int	8	1	0 to 255
long int	32	4	-2^{31} to $2^{31}-1$
unsigned long int	32	4	0 to $2^{32}-1$
float	32	4	3.4E-38 to 3.4E+38
double	64	8	1.7E-308 to 1.7E+308
long double	80	10	3.4E-4932 to 1.1E+4932

Fixed length binary tree

จะเห็นได้ว่าค่อนข้างจะสิ้นเปลืองจำนวนบิตในการจัดเก็บมาก เนื่องจากมีตัวอักษรที่แตกต่างกันในไฟล์เพียง 6 ตัวเท่านั้น จึงเป็นไปได้ว่า เราสามารถลดจำนวนบิตลงให้สัมพันธ์กับความแตกต่างของตัวอักษรในไฟล์ได้

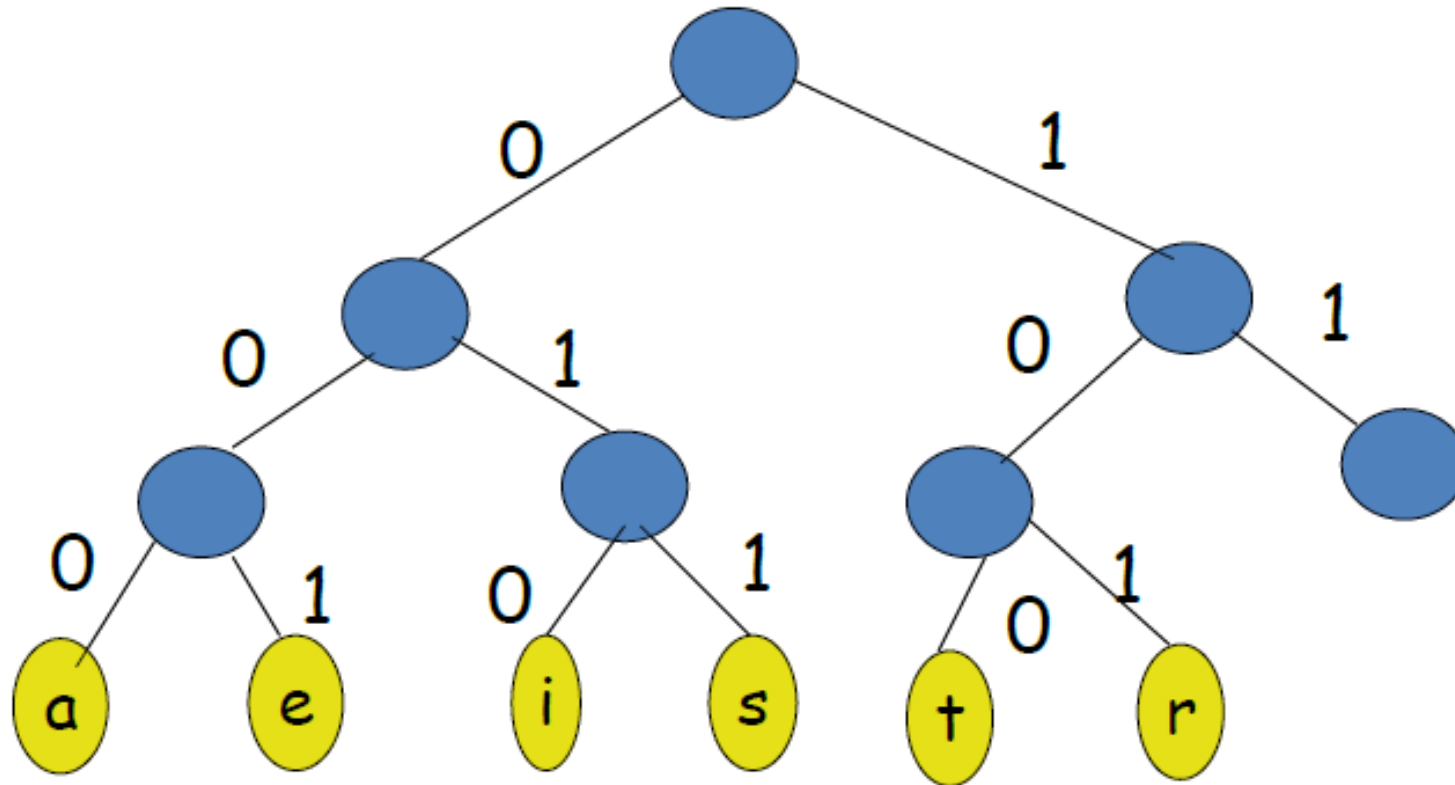
$$n \leq 2^{bit}$$

Character	Code	Frequency
a	000	10
e	001	15
i	010	12
s	011	3
t	100	4
r	101	1

TOTAL $45 * 3 = 135$ bits

Fixed length binary tree

เราสามารถแสดงต้นไม้ของรหัสไบนารี เพื่อความรวดเร็วสำหรับการถอดรหัสดังนี้



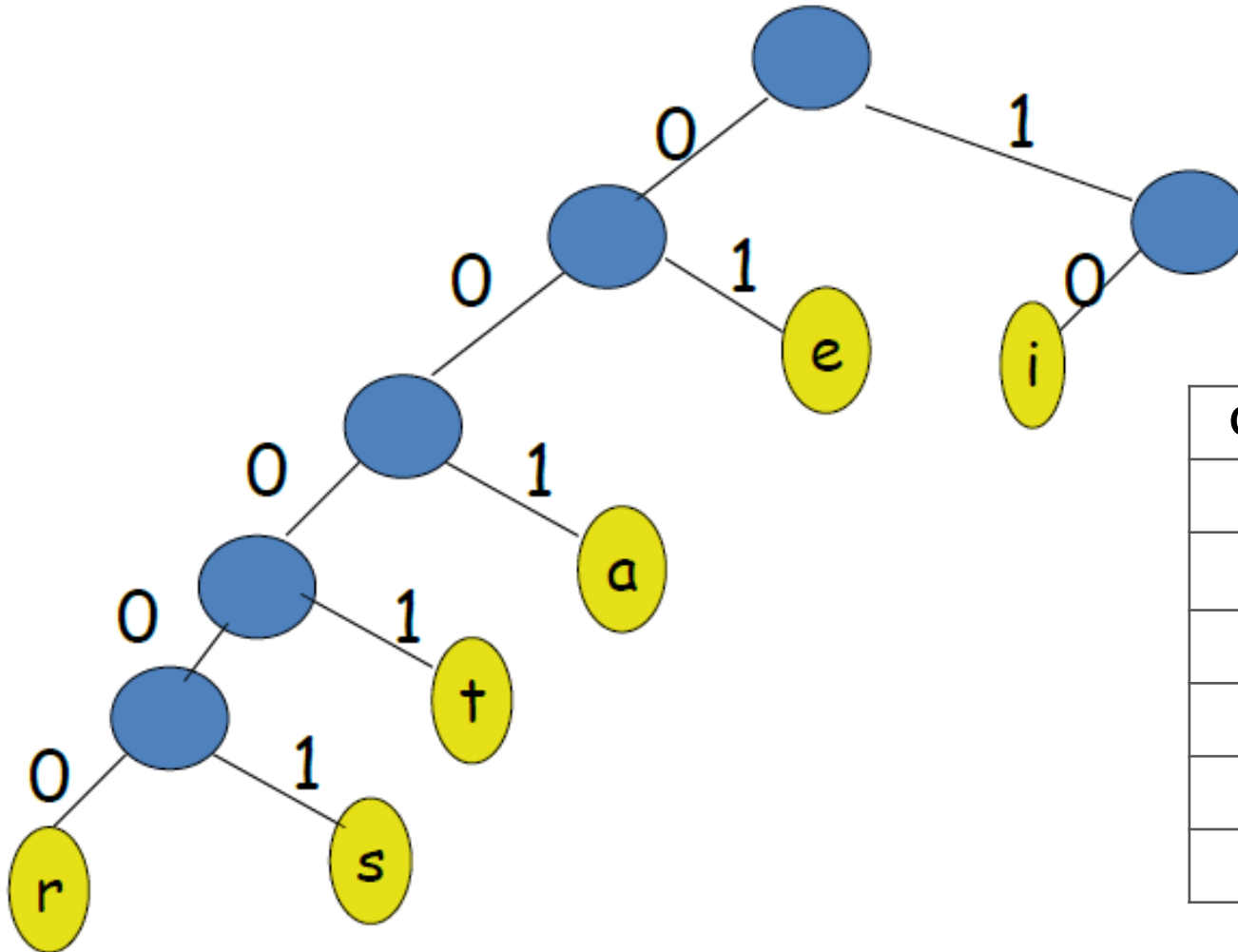
Variable-length binary coding

อย่างไรก็ตามถึงแม้ว่าจะลดพื้นที่ในการจัดเก็บได้ แต่ก็ยังคงไม่ optimal อยู่ดี ลองพิจารณาตัวอย่างด้านล่าง

Character	Code	Frequency
a	000	10
e	11	15
i	010	12
s	011	3
t	100	4
r	101	1

TOTAL $(30 \times 3) + (15 \times 2) = 90 + 30 = 120$ bits

Variable-length binary coding



Character	Code	Frequency
a	001	10
e	01	15
i	10	12
s	00001	3
t	0001	4
r	00000	1

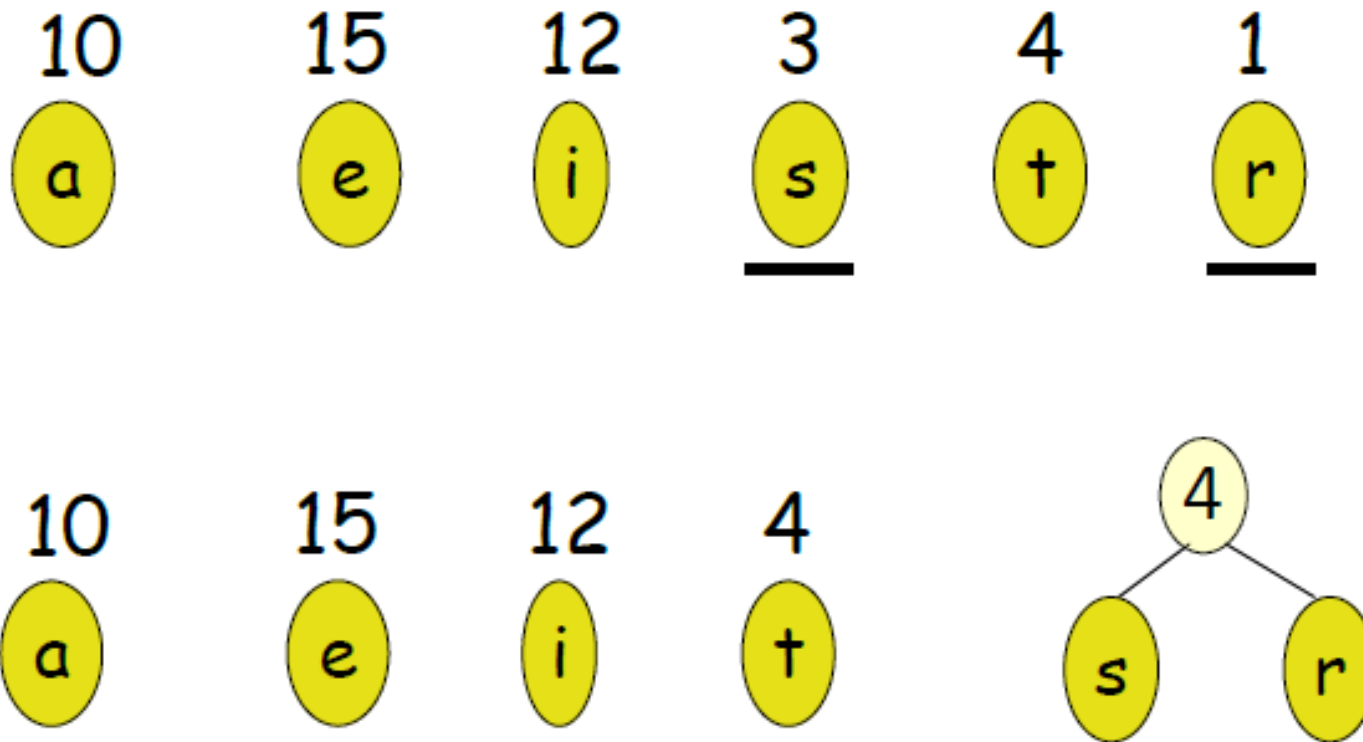
TOTAL = 112 bits

Huffman Coding

Letter ↕	Relative frequency in the English language ↕	
a	8.167%	
b	1.492%	
c	2.782%	
d	4.253%	
e	12.702%	
f	2.228%	
g	2.015%	
h	6.094%	
i	6.966%	
j	0.153%	
k	0.772%	
l	4.025%	
m	2.406%	
n	6.749%	
o	7.507%	
p	1.929%	
q	0.095%	
r	5.987%	
s	6.327%	
t	9.056%	
u	2.758%	
v	0.978%	
w	2.360%	
x	0.150%	
y	1.974%	
z	0.074%	

1. เริ่มจากนำทุกตัวอักษรและความถี่ หรือความน่าจะเป็น (probability) ก็ได้ ของมันมาสร้างเป็นไหนด
2. หยิบไหนด 2 ตัวที่มีความถี่ต่ำสุดมาเชื่อมกัน โดยสร้างเป็นต้นไม้หลัก
3. นำไหนดที่มีความถี่น้อยสุดจากไหนดที่เหลือมาเชื่อมกับต้นไม้ย่อยดังกล่าว แต่หากไหนดที่เหลือมีค่าความถี่น้อยกว่าต้นไม้หลัก ให้รวมกับไหนดอื่นเพื่อสร้างต้นไม้ย่อย จากนั้นจึงรวมต้นไม้ย่อยและต้นไม้หลักเข้าด้วยกัน
4. ทำเช่นนี้ไปเรื่อย ๆ จนกระทั่งต้นไม้ย่อยทั้งหมดรวมเป็นต้นไม้ต้นเดียว

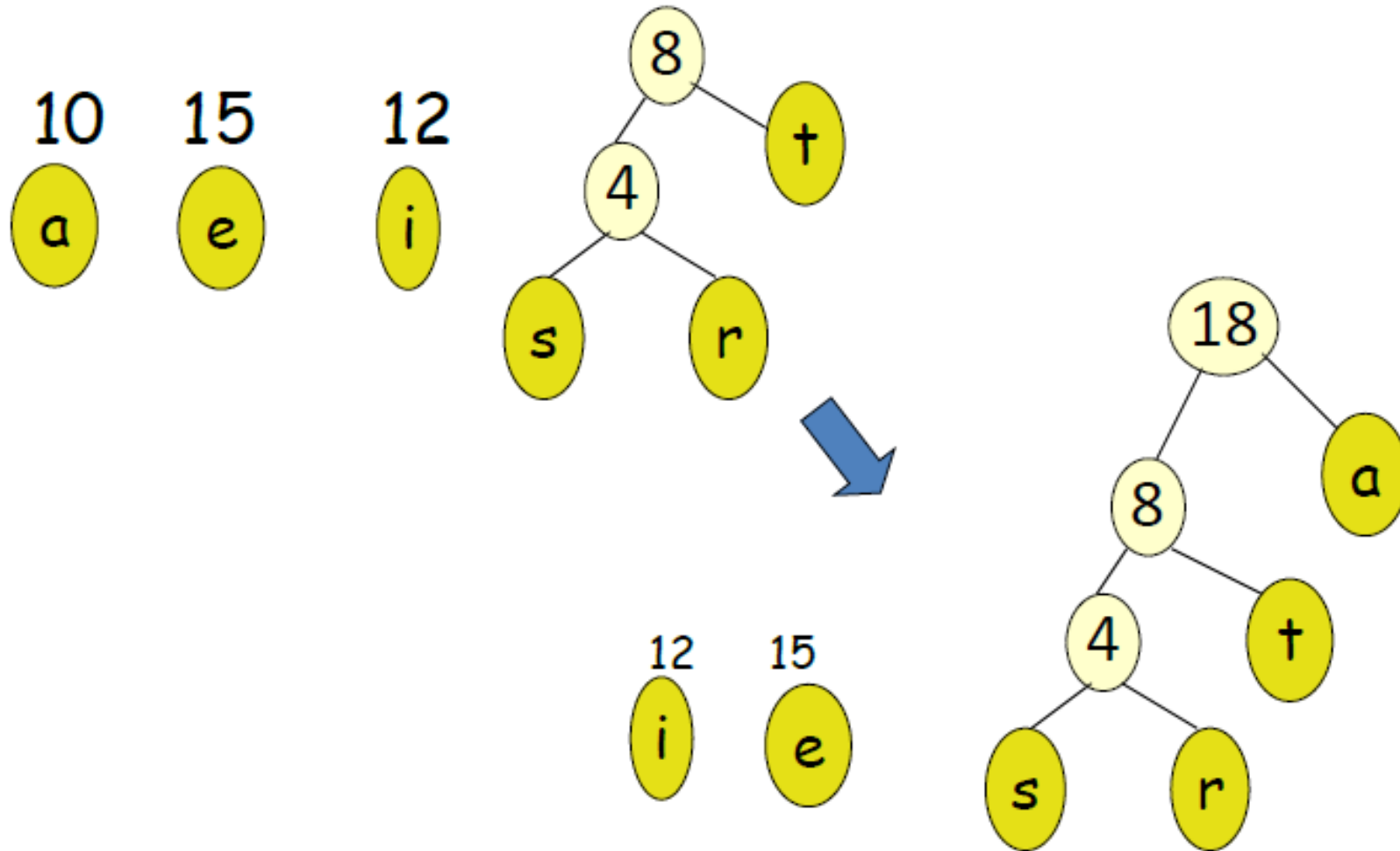
Example: Huffman Coding



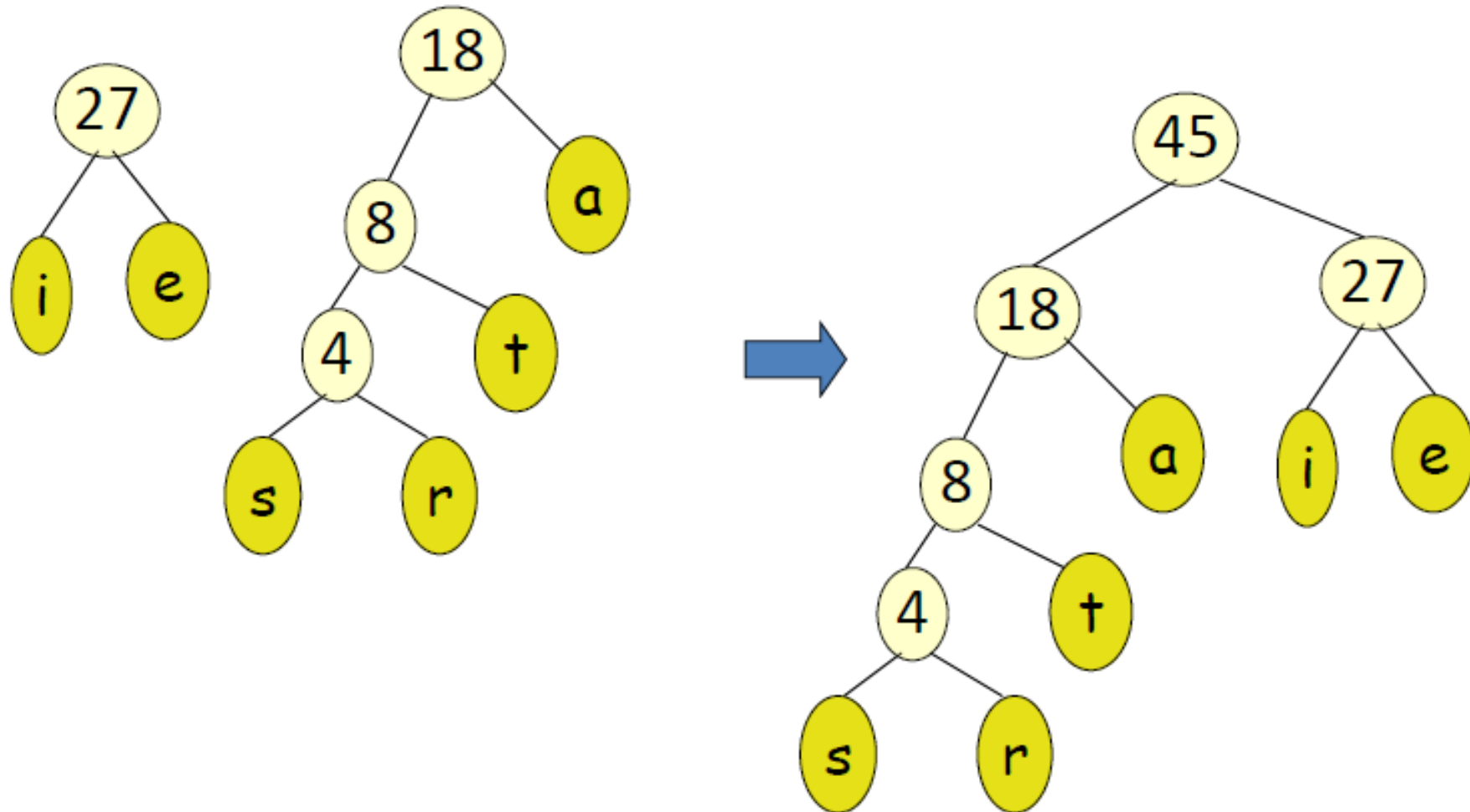
Character	Frequency
a	10
e	15
i	12
s	3
t	4
r	1

โหนดที่จะนำมารวมกับต้นไม้หลักต้องมีความถี่เท่ากันหรือมากกว่าต้นไม้หลัก

Example: Huffman Coding

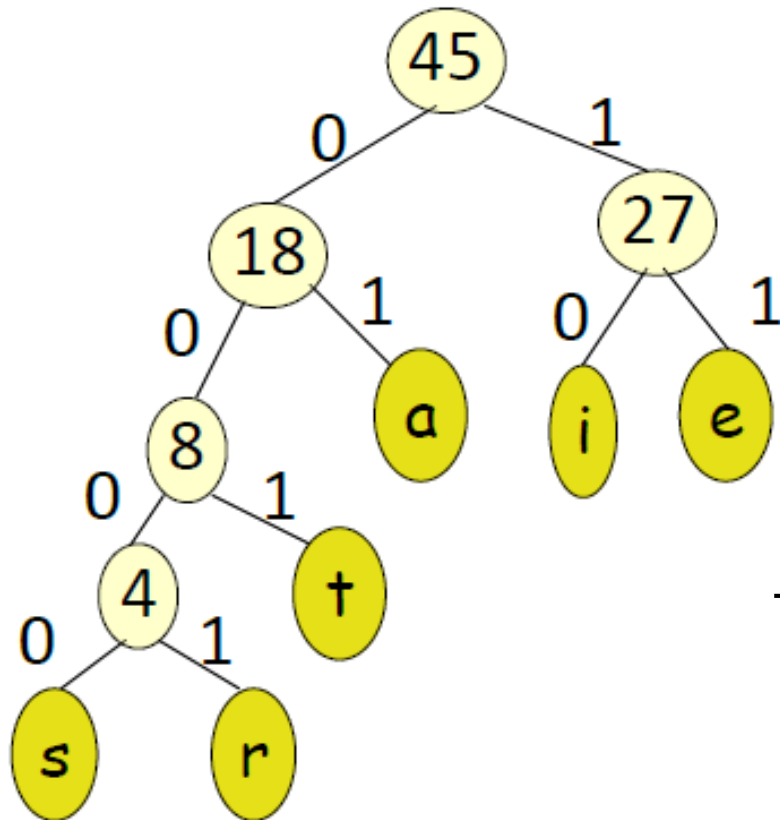


Example: Huffman Coding



Example: Huffman Coding

ให้กิ่งซ้ายมีค่าเป็น 0 และกิ่งขวามีค่าเป็น 1 จากนั้นรหัสของแต่ละตัวอักษรจะเริ่มไล่จาก root node



Character	Code	Frequency
a	01	10
e	11	15
i	10	12
s	0000	3
t	001	4
r	0001	1

$$\begin{aligned}
 \text{TOTAL} &= (10 \times 2) + (2 \times 15) + (2 \times 12) + (4 \times 3) + (3 \times 4) + (4 \times 1) \\
 &= 20 + 30 + 24 + 12 + 12 + 4 \\
 &= \mathbf{102 \text{ bits}}
 \end{aligned}$$

Huffman coding in C

- โค้ดภาษาซีของ Huffman coding <http://www.programminglogic.com/implementing-huffman-coding-in-c/>
- ถูกนำไปใช้งานจริงในแอปพลิเคชันสำหรับการบีบอัดไฟล์ PKZIP และการเข้ารหัสไฟล์ multimedia MP3